Introduction to Kubernetes

Before Kubernetes, Docker and Docker Swarm transformed how developers package applications. It allowed them to bundle an application and its dependencies into a single, portable unit called a **container**. This worked fine for small-scale deployments, but when applications grew to hundreds or thousands of containers the following problems appeared:

- Scalability Issues
- Multi-Cloud Deployments
- Security & Resource Management
- Rolling Updates & Zero Downtime Deployments

This is the problem Kubernetes was created to solve. It acts as the "brain" or **orchestrator** for your containers, handling the complex task of managing them at scale automatically.

Kubernetes

Kubernetes**,** often shortened to K8s (K, 8 letters, s), is an open-source platform that automates the deployment, scaling**,** and management of containerized applications.

- **Origin**: Developed by Google, inspired by internal systems Borg and Omega.
- **Launch**: Officially released in 2014.
- **CNCF Donation**: Donated to the Cloud Native Computing Foundation (CNCF) in 2015, which now maintains it.
- **Adoption**: Widely used across major cloud providers today.
- **Name Meaning**: *Kubernetes* comes from Greek, meaning **“**helmsman” or “pilot**”**, symbolizing its role in steering applications.

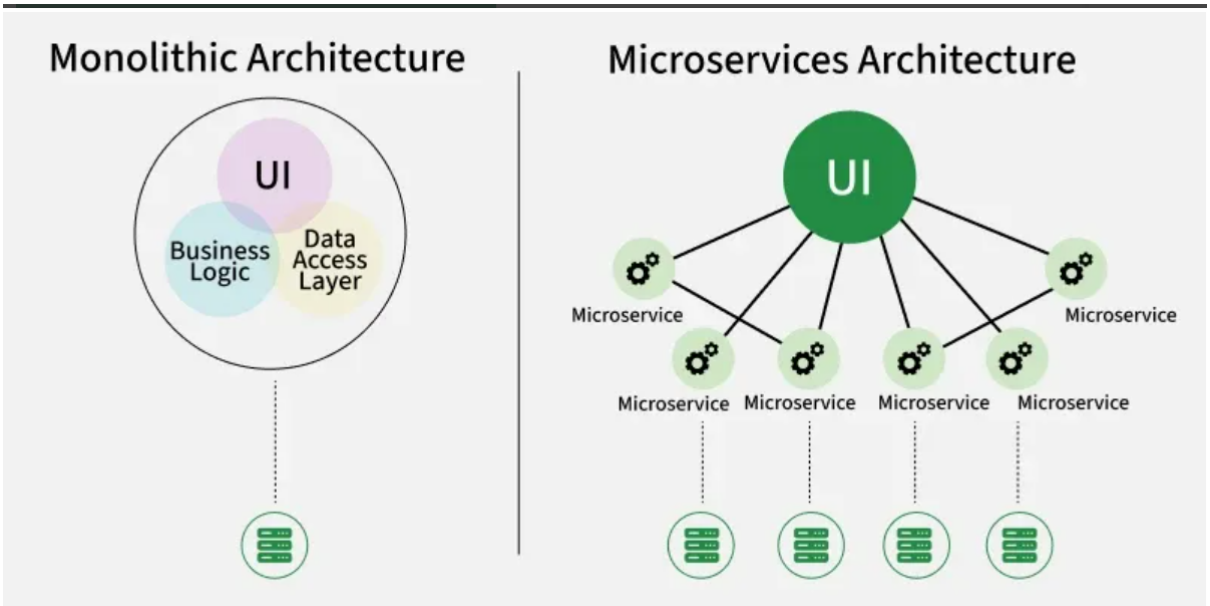
Think of Kubernetes as an orchestra conductor. Each container is a musician. While you can manage a few musicians yourself, you need a conductor to coordinate an entire orchestra to play a complex symphony. You simply give the conductor the sheet music (your desired configuration), and they ensure every musician plays their part correctly, replacing someone who falls ill and bringing in more players.

Some features of K8s:

- **Automated Scheduling** – Efficiently places containers on nodes for optimal resource use.
- **Self-Healing** – Automatically restarts, replaces, and reschedules failed containers.
- **Rollouts & Rollbacks** – Manages application updates and reverts when needed.
- **Scaling & Load Balancing** – Supports horizontal scaling and distributes traffic.
- **Resource Optimization** – Monitors and ensures efficient resource utilization.

Monolithic Vs Microservices

In the past, applications were built using a monolithic architecture, where everything was interconnected and bundled into one big codebase. This made updates risky for example, if you wanted to change just the payment module in an e-commerce app, you had to redeploy the entire application. A small bug could crash the whole system.



To overcome this, the industry moved toward microservices**,** where each feature (like payments, search, or notifications) is built and deployed independently. This made applications more flexible and scalable.

But with **microservices** came a new challenge instead of running one big app, companies now had to manage hundreds or thousands of small containerized services. Containers solved the packaging problem, but without a way to orchestrate them, things got messy. That's where Kubernetes came in acting like a smart manager that automates deployment, scaling, and coordination of all those microservices.

Terminologies in K8s

Think of Kubernetes as a well-organized company where different teams and systems work together to run applications efficiently. Here's how the key terms fit into this system:

1. Pod

A Pod is the smallest unit you can deploy in Kubernetes. It wraps one or more containers that need to run together, sharing the same network and storage. Containers inside a Pod can easily communicate and work as a single unit.

2. Node

A Node is a machine (physical or virtual) in a Kubernetes cluster that runs your applications. Each Node contains the tools needed to run Pods, including the container runtime (like Docker), the Kubelet (agent), and the Kube proxy (networking).

3. Cluster

A Kubernetes cluster is a group of computers (called nodes) that work together to run your containerized applications. These nodes can be real machines or virtual ones.

There are two types of nodes in a Kubernetes cluster:

1. **Master node (Control Plane):**
    - Think of it as the brain of the cluster.
    - It makes decisions, like where to run applications, handles scheduling, and keeps track of everything.
1. **Worker nodes:**
    - These are the machines that actually run your apps inside containers.

- Each worker node has a Kubelet (agent), a container runtime (like Docker or containerd), and tools for networking and monitoring.

4. Deployment

A Deployment is a Kubernetes object used to manage a set of Pods running your containerized applications. It provides declarative updates, meaning you tell Kubernetes what you want, and it figures out how to get there.

5. ReplicaSet

A ReplicaSet ensures that the right number of identical Pods are running.

6. Service

A Service in Kubernetes is a way to connect applications running inside your cluster. It gives your Pods a stable way to communicate, even if the Pods themselves keep changing.

7. Ingress

Ingress is a way to manage external access to your services in a Kubernetes cluster. It provides HTTP and HTTPS routing to your services, acting as a reverse proxy.

8. ConfigMap

A ConfigMap stores configuration settings separately from the application, so changes can be made without modifying the actual code.

Imagine you have an application that needs some settings, like a database password or an API key. Instead of hardcoding these settings into your app, you store them in a ConfigMap. Your application can then read these settings from the ConfigMap at runtime, which makes it easy to update the settings without changing the app code.

9. Secret

A Secret is a way to store sensitive information (like passwords, API keys, or tokens) securely in a Kubernetes cluster.

10. Persistent Volume (PV)

A Persistent Volume (PV) in Kubernetes is a piece of storage in the cluster that you can use to store data and it doesn't get deleted when a Pod is removed or restarted.

11. Kubelet

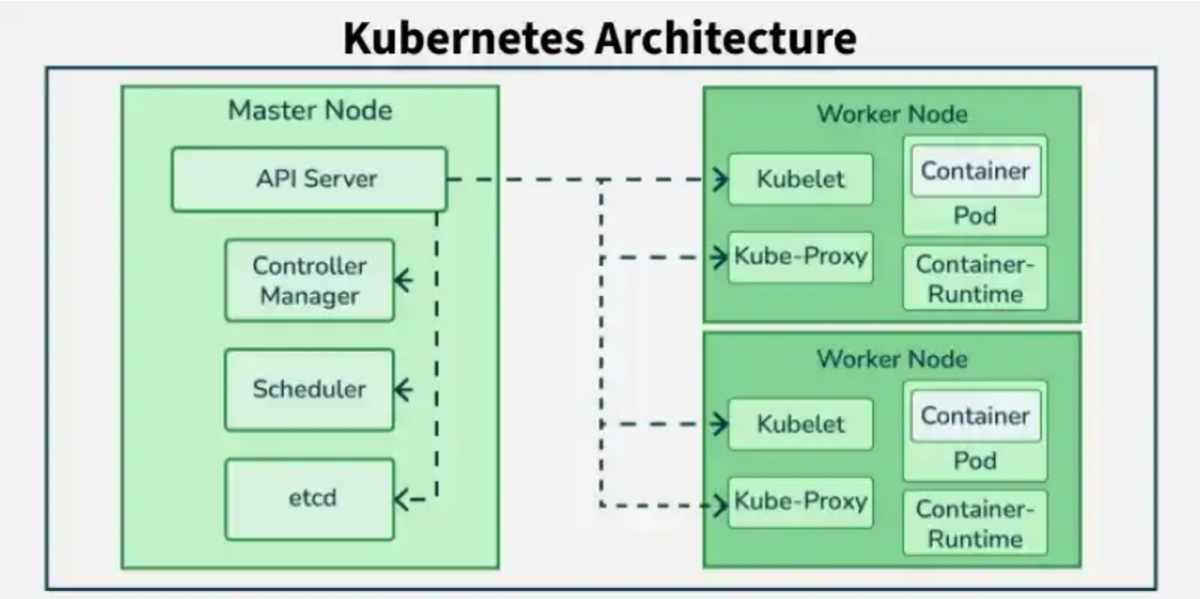
A Kubelet runs on each Worker Node and ensures Pods are running as expected.

12. Kube-proxy

Kube-proxy manages networking inside the cluster, ensuring different Pods can communicate.

Kubernetes - Architecture

Kubernetes follows a client–server architecture consisting of a control plane (master) and worker nodes. The control plane includes components such as the API Server, Scheduler, Controller Manager, and etcd for cluster state storage.



Worker nodes run application workloads and include kubelet for communication with the control plane, kube-proxy for networking, and a container runtime (such as containerd or Docker) to manage containers.

Control Plane Components

It is basically a collection of various components that help us in managing the overall health of a cluster. For example, if you want to set up new pods, destroy pods, scale pods, etc. Basically, 4 services run on the Control Plane:

1. Kube-API server

The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. It is like an initial gateway to the cluster that listens to updates or queries via CLI like Kubectl.

- Kubectl communicates with API Server to inform what needs to be done like creating pods or deleting pods etc. It also works as a gatekeeper.
- It generally validates requests received and then forwards them to other processes.
- No request can be directly passed to the cluster, it has to be passed through the API Server.

2. Kube-Scheduler

When API Server receives a request for Scheduling Pods then the request is passed on to the Scheduler. It intelligently decides on which node to schedule the pod for better efficiency of the cluster.

3. Kube-Controller-Manager

The kube-controller-manager is responsible for running the controllers that handle the various aspects of the cluster's control loop. These controllers include the replication controller, which ensures that the desired number of replicas of a given application is running, and the node controller, which ensures that nodes are correctly marked as "ready" or "not ready" based on their current state.

4. etcd

It is a key-value store of a Cluster. The Cluster State Changes get stored in the etcd. It acts as the Cluster brain because it tells the Scheduler and other processes about which resources are available and about cluster state changes.

Worker Node Components

These are the nodes where the actual work happens. Each Node can have multiple pods and pods have containers running inside them. There are 3 processes in every Node that are used to Schedule and manage those pods. The following are the some of the components related to Node:

1. Container runtime

A container runtime is the software responsible for running containers on a node. It executes the application containers that are part of pods and manages their lifecycle. Popular container runtimes include Docker, containerd, and CRI-O.

2. kubelet

The kubelet is an agent that runs on each Kubernetes node. It communicates with both the container runtime and the Kubernetes control plane to ensure that containers within pods are running as specified. The kubelet monitors the state of pods, reports node and pod status to the API server, and ensures that the desired container configuration is maintained.

3. kube-proxy

The kube-proxy runs on each node and manages network communication for pods. It implements Kubernetes Services by maintaining network rules that allow pods to communicate with each other and with external clients. Kube-proxy can handle traffic via iptables or IPVS, enabling service discovery and load balancing across pods.

Addons Plug-in

Kubernetes add-ons are plug-ins that enhance the cluster's functionality, often installed as Kubernetes resources like DaemonSets, Deployments, and more. These add-ons are typically deployed within the kube-system namespace, providing cluster-level capabilities and extending the native features of Kubernetes.

1. **CoreDNS**: A flexible, extensible DNS server that provides name resolution services for Kubernetes clusters, ensuring efficient service discovery and network routing.

1. **KubeVirt**: Allows the running of virtual machines alongside containers, providing a unified management platform for both VMs and containerized applications.

1. **ACI (Application Containerization Interface)**: Facilitates the integration and management of containers across different environments, improving the portability and scalability of applications.

1. **Calico**: A network policy engine that provides secure, high-performance networking for Kubernetes clusters, supporting both network policy enforcement and advanced routing capabilities.

Design of pods

A Pod is the smallest deployable unit in Kubernetes, representing a single instance of a running process in your cluster. Its design focuses on creating a shared execution environment for one or more tightly coupled containers.

Core Architectural Principles

- **Atomic Unit:** Pods are created, scheduled, and managed as a single entity. If a container within a Pod fails, Kubernetes manages the Pod based on its restartPolicy.
- **Shared Context:** All containers in a Pod are guaranteed to be co-located on the same Worker Node and share a common environment.
- **Ephemeral Nature:** Pods are disposable; they are not "rescheduled" if a node fails but are instead replaced by a new, near-identical Pod with a different UID.

Shared Resources in a Pod

Containers within a single Pod share several key Linux namespaces:

- **Network Namespace:** Every Pod is assigned a unique IP address. Containers within the same Pod communicate with each other via `localhost` and share the same port space.
- **Storage Volumes:** Pods can define Volumes that all containers in the Pod can mount and access simultaneously for data exchange or persistence.
- **Pause Container:** A hidden Pause container is automatically created for every Pod to "hold" the shared namespaces (like Network and IPC) even if other application containers restart.

Multi-Container Design Patterns

While most Pods run a single container, multi-container Pods use specific patterns to extend functionality without modifying application code:

- **Sidecar:** A helper container that runs alongside the main app to provide auxiliary features like logging, monitoring, or data synchronization.

- **Init Containers:** Specialized containers that run and complete sequentially *before* the main app containers start (e.g., for setting up database schemas or cloning Git repos).
- **Ambassador:** A proxy that handles the main container's connection to the outside world.
- **Adapter:** Standardizes the output of the main container (e.g., transforming logs) for external monitoring tools.

Management and Scheduling

- **Controllers:** In production, Pods are rarely created directly.
  Instead, Controllers like **Deployments**, **StatefulSets**,
  or **DaemonSets** manage their lifecycle, scaling, and replacement.
- **Scheduling:** The kube-scheduler assigns Pods to nodes based on Resource Requests and Limits (CPU/Memory) and constraints like Node Affinity.